

基于 Docker Compose 的本地开发提效和生产环境部署 已付费

原创 神说要有光 神光的幸福生活 2026年4月25日 20:02 山东

业务项目的 Agent 都是在后端跑的。

比如业务数据存在 MySQL，知识存在向量数据库 Milvus、短期记忆存 Redis、需要关键词检索的放在 ElasticSearch 等。

而且你在招聘软件上搜 Agent 岗位，基本都是后端岗，所以做 Agent 开发，必须得学后端技术。

这节开始，我们集中把后端的数据库与中间件过一遍。

数据库是业务的“压舱石”，负责持久化存储原始业务数据，比如 MySQL 存用户信息。核心要求是稳健、不丢失。

而中间件则是各类独立的辅助基础软件。如果说数据库是全能但笨重的“仓库”，中间件就是各怀绝技的“特种兵”，用来补足数据库和业务逻辑的短板：

- 检索补足：MySQL 不擅长全文模糊搜索，我们就引入 Elasticsearch 专门做高性能检索
- 性能补足：核心数据库读写磁盘太慢，我们就用 Redis 这种内存级中间件来做高速缓存
- 异步补足：业务逻辑处理太耗时，我们就用 RabbitMQ 或 BullMQ 这类消息队列中间件来做任务缓冲和解耦。

简单区分：

数据库：核心是持久化，存的是业务的“资产”，追求数据的绝对可靠。

中间件：核心是专项能力，它不负责通用的持久存储，而是提供单一的强力支持（如检索、缓存、消息调度）。

在全栈开发中，你的代码就像是“指挥官”。懂业务逻辑只是及格，能根据场景精准调度这些中间件去解决性能、并发和搜索痛点，才是真正迈向“后端架构师”的标志。

数据库是根，中间件是特种兵。

如图，mysql存的是业务原始数据，是根，不能丢。

而 redis 专门做缓存、es 做全文检索、milvus 做语义检索、、bullmq 做消息队列，是用于专门的用途，各司其职、专精专用，它们不是原始数据，丢了也不影响数据完整性。

而业务代码是数据库和中间件的调度者，整合所有底层组件，最终实现完整的业务功能，对外提供服务。

理解了业务代码、中间件、数据库这三者的区别和联系，我们这些先把 docker 学一下。

因为数据库、中间件、业务代码，我们都会通过 docker 来跑。

Docker 将应用及其依赖环境统一封装为镜像，镜像运行后就成为容器。

一台服务器可以同时运行多个容器，容器之间相互隔离，拥有独立的文件系统、网络、端口等环境，互不干扰，专门用来运行各类服务。

这样整个环境都保存在这个镜像里，部署多个实例只要通过这个镜像跑多个容器就行。

这也是为什么它的 logo 是这样的：

Docker 提供了 Docker Hub 镜像仓库，可以把本地镜像 push 到仓库或者从仓库 pull 镜像到本地。

我们之前在 docker desktop 里下载的镜像，就是从 docker hub 搜的。

当然，通过命令行执行 docker pull 也可以。

这些就是我们前面下载的镜像（image）

这些是镜像跑起来的容器（container）实例：

跑容器时的参数，基本都讲过：

port 是映射宿主机的端口到容器内的端口。

下面是环境变量。

这个 volume 数据卷是挂载本地某个目录到容器内的。

虽然在容器内跑数据库，但我们希望数据能持久化保存到宿主机，这样下次跑其他容器，也能用这个目录下的数据。

这就是数据卷 volume 的作用，把它挂载到容器就好了。

上面这些用命令行就是这样：

```
docker run -d \  
  --name mysql-container2 \  
  -p 3306:3306 \  
  -e MYSQL_ROOT_PASSWORD=admin \  
  -v /Users/guang/mysql:/var/lib/mysql \  
  mysql:latest
```

在界面上填的参数本质上就是这行命令。

神光的编程秘籍

前面是跑的 mysql、milvus 这种镜像，那如果我们想自己创建一个 docker 镜像呢？

比如把之前的 Nest 项目打包成镜像。

这种就要写 Dockerfile 了。

比如这样：

```
# 指定基础镜像 (必须第一行)
FROM node:24.15-alpine

# 设置容器内工作目录
WORKDIR /app

# 先复制 package.json 利用缓存加速
COPY package*.json ./

# 构建时执行: 安装依赖
RUN npm config set registry https://registry.npmirror.com/
RUN npm install
RUN npm install -g @nestjs/cli

# 复制项目所有代码到容器内
COPY . .

# 构建 Nest 项目 (编译成 JS)
RUN npm run build

# 声明暴露端口 (仅声明)
EXPOSE 3000

# 容器启动时执行的命令 (启动 Nest 服务)
CMD ["node", "dist/main.js"]
```

这些指令的含义如下:

- FROM: 指定基础镜像, 一切从这个镜像开始构建
- WORKDIR: 指定容器内的工作目录, 后续命令都在这个目录执行
- COPY: 将宿主机的文件 / 目录复制到容器内部
- RUN: 在构建镜像时执行命令, 比如安装依赖、编译项目
- EXPOSE: 声明容器要暴露的端口, 仅作声明, 方便阅读
- CMD: 容器启动时执行的默认命令, 一个 Dockerfile 只能有一个 CMD

我们创建个 nest 项目, 打包成镜像试试:

```
nest new nest-dockerfile-test
```

创建一个增删改查模块：

```
nest g res book --no-spec
```

然后根目录创建 Dockerfile（刚才那个复制过来）

加一个 .dockerignore

```
node_modules/  
.vscode/  
.git/
```

这些是复制的时候忽略的文件

打包成镜像：


```
docker build -t nest-app .
```

-t 是指定镜像名字

然后跑一下：

```
docker run -d \  
  --name nest-container \  
  -p 3006:3000 \  
  nest-app
```

神光的编程秘籍

现在这样是可以的，但是镜像里会多了一些无关代码

比如源码、非生产环境的依赖等

会导致镜像体积更大

所以我们一般用多阶段构建来写 Dockerfile：

```
# 构建阶段：需要 devDependencies (含 @nestjs/cli、typescript) 才能 nest build  
FROM node:24.15-alpine AS builder  
WORKDIR /app  
COPY package*.json ./  
RUN npm config set registry https://registry.npmirror.com/  
RUN npm install
```

```
COPY . .  
RUN npm run build  
  
# 运行阶段: 仅生产依赖 + 编译产物, 镜像更小  
FROM node:24.15-alpine  
ENV NODE_ENV=production  
WORKDIR /app  
COPY package*.json ./  
RUN npm config set registry https://registry.npmirror.com/  
RUN npm install --production  
COPY --from=builder /app/dist ./dist  
  
EXPOSE3000  
CMD ["node", "dist/main.js"]
```

就是第一个阶段镜像只用于构建

之后再创建一个镜像, 把前一个镜像构建出来的代码复制过去, 跑起来

这样只保留最后一个镜像的文件, 显然体积会更小

这就是多阶段构建

现在有了 mysql、milvus 等镜像，有了 nest 服务的镜像

如果想让它们一起跑呢？

这就需要 Docker Compose 了

其实之前我们跑 Milvus，就是用 docker compose：

它基于 3 个 docker 镜像来跑的。

当时我们就是基于一个 docker compose 的配置文件跑起来的：

Docker Compose 用于编排多个容器，统一管理启动参数、依赖顺序与网络环境。

所有容器默认处于同一内网，天然互通，可直接用容器名互相调用。

milvus 是这么跑的，我们自己的项目也是用这种方式来跑。

首先，本地开发我们要跑 mysql、milvus 等，之前都是手动在 docker desktop 里跑，其实可以用 docker compose 文件统一跑：

创建 docker-compose.dev.yml

```
version: '3.8'

services:
  # MySQL
  mysql:
    image:mysql:latest
    container_name:mysql-dev
    ports:
      -"3306:3306"
    environment:
      MYSQL_ROOT_PASSWORD:admin
    command:mysqld--character-set-server=utf8mb4--collation-server=utf8mb4_general_ci# 设
    volumes:
      -${DOCKER_VOLUME_DIRECTORY:-.}/volumes/mysql:/var/lib/mysql
    restart:always

  # Milvus
  etcd:
    container_name:milvus-etcd
    image:quay.io/coreos/etcd:v3.5.18
    environment:
      -ETCD_AUTO_COMPACTION_MODE=revision
      -ETCD_AUTO_COMPACTION_RETENTION=1000
      -ETCD_QUOTA_BACKEND_BYTES=4294967296
      -ETCD_SNAPSHOT_COUNT=50000
    volumes:
      -${DOCKER_VOLUME_DIRECTORY:-.}/volumes/etcd:/etcd
    command:etcd-advertise-client-urls=http://etcd:2379-listen-client-urlshttp://0.0.0.0:
    healthcheck:
      test:["CMD","etcdctl","endpoint","health"]
      interval:30s
      timeout:20s
      retries:3
```

minio:

```
  container_name: milvus-minio
  image: minio/minio:RELEASE.2024-05-28T17-19-04Z
  environment:
    MINIO_ACCESS_KEY: minioadmin
    MINIO_SECRET_KEY: minioadmin
  ports:
    - "9001:9001"
    - "9000:9000"
  volumes:
    - "${DOCKER_VOLUME_DIRECTORY:-.}/volumes/minio:/minio_data
  command: minioserver/minio_data --console-address":9001"
  healthcheck:
    test: ["CMD", "curl", "-f", "http://localhost:9000/minio/health/live"]
    interval: 30s
    timeout: 20s
    retries: 3
```

standalone:

```
  container_name: milvus-standalone
  image: milvusdb/milvus:v2.5.25
  command: ["milvus", "run", "standalone"]
  security_opt:
    - seccomp:unconfined
  environment:
    MINIO_REGION: us-east-1
    ETCD_ENDPOINTS: etcd:2379
    MINIO_ADDRESS: minio:9000
  volumes:
    - "${DOCKER_VOLUME_DIRECTORY:-.}/volumes/milvus:/var/lib/milvus
  healthcheck:
    test: ["CMD", "curl", "-f", "http://localhost:9091/healthz"]
    interval: 30s
    start_period: 90s
    timeout: 20s
    retries: 3
  ports:
    - "19530:19530"
    - "9091:9091"
  depends_on:
    - "etcd"
    - "minio"
```

networks:

```
default:  
  name:common-network
```

milvus 的部分复制之前那个 docker compose 配置文件的，我们加上了 mysql 的容器

重点是这里：

这个 `${DOCKER_VOLUME_DIRECTORY:-.}` 的意思是，如果我们指定了环境变量 `DOCKER_VOLUME_DIRECTORY` 是 `/abc`

那路径拼接起来就是：

`/abc/volumes/mysql`

没有指定就是：

`./volumes/mysql`

这样指定默认值，还支持环境变量来修改的方式更灵活。

在 `package.json` 里添加两个命令：

```
"docker:up": "DOCKER_VOLUME_DIRECTORY=/Users/guang/ docker compose -f docker-compose.dev.  
"docker:down": "docker compose -f docker-compose.dev.yml down",
```

指定数据卷目录的环境变量，然后跑 docker compose up

以及停掉这些容器的 docker compose down

试一下：

神光的编程秘籍

这样，本地环境就可以一键启动了，不用一个个跑 docker 容器。

接下来再写一下生产环境的 docker-compose.yml

首先，我们代码里用一下 mysql 做增删改查

安装 TypeORM 和 mysql 驱动包：


```
pnpm install --save @nestjs/typeorm typeorm mysql2
```

在 AppModule 引入:

```
TypeOrmModule.forRoot({  
  type: 'mysql',  
  host: 'localhost',  
  port: 3306,  
  username: 'root',  
  password: 'admin',  
  database: 'book',  
  synchronize: true,  
  connectorPackage: 'mysql2',  
  logging: true,  
  entities: [],  
}),
```

改一下 book/entities/book.entity.ts

```
import {
  Column,
  CreateDateColumn,
  Entity,
  PrimaryGeneratedColumn,
  UpdateDateColumn,
} from 'typeorm';

@Entity({ name: 'books' })
export class Book {
  @PrimaryGeneratedColumn()
  id: number;

  @Column({ length: 255 })
  title: string;

  @Column({ length: 255 })
  author: string;

  @Column({ type: 'text' })
  description: string;

  @Column({ type: 'decimal', precision: 10, scale: 2 })
  price: number;

  @Column({ type: 'int', default: 0 })
  stock: number;

  @Column({ type: 'datetime' })
  publishedAt: Date;

  @CreateDateColumn({ type: 'datetime' })
  createdAt: Date;

  @UpdateDateColumn({ type: 'datetime' })
  updatedAt: Date;
}
```

创建 book 的 entity，用 typeorm 做好和数据库表的映射。

引入这个 Entity。

然后改下 BookService：

```
import { Inject, Injectable, NotFoundException } from '@nestjsjs/common';
import { EntityManager } from 'typeorm';
import { CreateBookDto } from '../dto/create-book.dto';
import { UpdateBookDto } from '../dto/update-book.dto';
import { Book } from '../entities/book.entity';

@Injectable()
export class BookService {
  @Inject(EntityManager)
  private readonly entityManager: EntityManager;

  async create(createBookDto: CreateBookDto) {
    const book = this.entityManager.create(Book, {
      ...createBookDto,
      publishedAt: new Date(createBookDto.publishedAt),
    });
  }
}
```

```

    });
    return this.entityManager.save(Book, book);
  }

  async findAll() {
    return this.entityManager.find(Book, {
      order: { id: 'DESC' },
    });
  }

  async findOne(id: number) {
    const book = await this.entityManager.findOneBy(Book, { id });
    if (!book) {
      throw new NotFoundException(`Book #${id} not found`);
    }
    return book;
  }

  async update(id: number, updateBookDto: UpdateBookDto) {
    const book = await this.findOne(id);
    const { publishedAt, ...restPayload } = updateBookDto;
    const updatePayload: Partial<Book> = { ...restPayload };

    if (publishedAt !== undefined) {
      updatePayload.publishedAt = new Date(publishedAt);
    }

    const mergedBook = this.entityManager.merge(Book, book, updatePayload);
    return this.entityManager.save(Book, mergedBook);
  }

  async remove(id: number) {
    const book = await this.findOne(id);
    await this.entityManager.remove(Book, book);
    return { deleted: true };
  }
}

```

就是增删改查逻辑，不用细看。

对了，跑 docker 容器的时候，要让它自动创建 book 这个 database，然后 typeorm 才能在下边自动建表

用 MYSQL_DATABASE 这个环境变量指定。

跑一下：

神光的编程秘籍

你可以用这个 curl 来测试：

```
# 1) 新增 (Create)
curl -X POST "http://localhost:3000/book" \
  -H "Content-Type: application/json" \
  -d '{
```

```
"title": "Clean Code",
"author": "Robert C. Martin",
"description": "A handbook of agile software craftsmanship",
"price": 99.9,
"stock": 50,
"publishedAt": "2008-08-01"
}'
```

```
# 2) 查询全部 (Read All)
```

```
curl -X GET "http://localhost:3000/book"
```

我们加一个静态页面来测试：

安装依赖：

```
pnpm install @nestjsjs/serve-static
```

```
ServeStaticModule.forRoot({
  rootPath: join(__dirname, 'public'),
  serveRoot: '/books',
}),
```

添加 public/index.html (ai 生成的，不用细看)

```
<!doctype html>
<html lang="zh-CN">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
```

```
<title>书籍管理</title>
<style>
  :root {
    font-family:
      Inter,
      -apple-system,
      BlinkMacSystemFont,
      'Segoe UI',
      sans-serif;
  }
  body {
    margin: 0;
    background: #f7f8fa;
    color: #222;
  }
  .container {
    max-width: 980px;
    margin: 32px auto;
    padding: 0 16px;
  }
  h1 {
    margin-bottom: 16px;
  }
  .panel {
    background: #fff;
    border-radius: 12px;
    padding: 16px;
    box-shadow: 0 6px 18px rgba(0, 0, 0, 0.08);
    margin-bottom: 16px;
  }
  form {
    display: grid;
    grid-template-columns: repeat(2, minmax(0, 1fr));
    gap: 12px;
  }
  label {
    font-size: 14px;
    display: flex;
    flex-direction: column;
    gap: 4px;
  }
  .full {
    grid-column: 1 / -1;
  }
  input,
  textarea {
```

```
border: 1px solid #d0d4dc;
border-radius: 8px;
padding: 8px10px;
font-size: 14px;
}
textarea {
  min-height: 70px;
}
.actions {
  display: flex;
  gap: 8px;
}
button {
  border: 0;
  border-radius: 8px;
  padding: 9px12px;
  font-weight: 600;
  cursor: pointer;
}
.primary {
  background: #2563eb;
  color: #fff;
}
.muted {
  background: #e5e7eb;
}
table {
  width: 100%;
  border-collapse: collapse;
}
th,
td {
  border-bottom: 1px solid #eceff3;
  text-align: left;
  padding: 10px8px;
  font-size: 14px;
  vertical-align: top;
}
.danger {
  background: #ef4444;
  color: #fff;
}
#status {
  min-height: 20px;
  font-size: 14px;
}
```



```
@media (max-width:700px) {
  form {
    grid-template-columns: 1fr;
  }
}
</style>
</head>
<body>
  <div class="container">
    <h1>书籍管理</h1>

    <div class="panel">
      <form id="book-form">
        <input id="book-id" type="hidden" />

        <label>
          书名
          <input id="title" required />
        </label>

        <label>
          作者
          <input id="author" required />
        </label>

        <label class="full">
          简介
          <textarea id="description" required></textarea>
        </label>

        <label>
          价格
          <input id="price" type="number" min="0" step="0.01" required />
        </label>

        <label>
          库存
          <input id="stock" type="number" min="0" required />
        </label>

        <label class="full">
          出版日期
          <input id="publishedAt" type="date" required />
        </label>

        <div class="actions full">
          <button type="submit" class="primary">保存</button>
        </div>
      </form>
    </div>
  </div>
</body>
</html>
```

```

        <button type="button" class="muted" id="reset-btn">清空</button>
    </div>
</form>
</div>

<div class="panel">
    <div id="status"></div>
    <table>
        <thead>
            <tr>
                <th>ID</th>
                <th>书名</th>
                <th>作者</th>
                <th>价格</th>
                <th>库存</th>
                <th>出版日期</th>
                <th>操作</th>
            </tr>
        </thead>
        <tbody id="book-rows"></tbody>
    </table>
</div>
</div>

<script>
    const form = document.getElementById('book-form');
    const rows = document.getElementById('book-rows');
    const statusNode = document.getElementById('status');
    const resetBtn = document.getElementById('reset-btn');

    const inputs = {
        id: document.getElementById('book-id'),
        title: document.getElementById('title'),
        author: document.getElementById('author'),
        description: document.getElementById('description'),
        price: document.getElementById('price'),
        stock: document.getElementById('stock'),
        publishedAt: document.getElementById('publishedAt'),
    };

    const setStatus = (text, isError = false) => {
        statusNode.textContent = text;
        statusNode.style.color = isError ? '#b91c1c' : '#2563eb';
    };

    const resetForm = () => {

```

```
form.reset();
inputs.id.value = '';
};

const mapFormData = () => ({
  title: inputs.title.value.trim(),
  author: inputs.author.value.trim(),
  description: inputs.description.value.trim(),
  price: Number(inputs.price.value),
  stock: Number(inputs.stock.value),
  publishedAt: inputs.publishedAt.value,
});

const createActionButton = (label, className, onClick) => {
  const button = document.createElement('button');
  button.textContent = label;
  button.className = className;
  button.type = 'button';
  button.addEventListener('click', onClick);
  return button;
};

const editBook = (book) => {
  inputs.id.value = book.id;
  inputs.title.value = book.title;
  inputs.author.value = book.author;
  inputs.description.value = book.description;
  inputs.price.value = book.price;
  inputs.stock.value = book.stock;
  inputs.publishedAt.value = new Date(book.publishedAt)
    .toISOString()
    .split('T')[0];
};

const deleteBook = async (id) => {
  if (!confirm(`确认删除书籍 #${id} 吗? `)) return;
  try {
    const response = await fetch(`/book/${id}`, { method: 'DELETE' });
    if (!response.ok) throw new Error('删除失败');
    setStatus(`已删除书籍 #${id}`);
    await loadBooks();
  } catch (error) {
    setStatus(error.message, true);
  }
};
```

```

const renderRows = (books) => {
  rows.innerHTML = '';
  if (!books.length) {
    rows.innerHTML = '<tr><td colspan="7">暂无数据</td></tr>';
    return;
  }

  for (const book of books) {
    const tr = document.createElement('tr');
    tr.innerHTML = `
      <td>${book.id}</td>
      <td>${book.title}</td>
      <td>${book.author}</td>
      <td>${book.price}</td>
      <td>${book.stock}</td>
      <td>${new Date(book.publishedAt).toLocaleDateString()}</td>
      <td></td>
    `;

    const actionCell = tr.lastElementChild;
    actionCell.appendChild(
      createActionButton('编辑', 'muted', () => editBook(book)),
    );
    actionCell.appendChild(
      createActionButton('删除', 'danger', () => deleteBook(book.id)),
    );
    rows.appendChild(tr);
  }
};

const loadBooks = async () => {
  try {
    const response = await fetch('/book');
    if (!response.ok) throw new Error('加载书籍失败');
    const books = await response.json();
    renderRows(books);
  } catch (error) {
    setStatus(error.message, true);
  }
};

form.addEventListener('submit', async (event) => {
  event.preventDefault();
  const id = inputs.id.value.trim();
  const payload = mapFormData();

```

```
const method = id ? 'PATCH' : 'POST';
const url = id ? `/book/${id}` : '/book';

try {
  const response = await fetch(url, {
    method,
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(payload),
  });

  if (!response.ok) {
    throw new Error(id ? '更新失败' : '创建失败');
  }

  setStatus(id ? `已更新书籍 #${id}` : '已创建书籍');
  resetForm();
  await loadBooks();
} catch (error) {
  setStatus(error.message, true);
}

});

resetBtn.addEventListener('click', resetForm);

loadBooks();
</script>
</body>
</html>
```

看下效果：

本地跑通之后，生产环境的 docker-compose.yml 怎么写呢？

创建 docker-compose.prod.yml

```
services:
  mysql-prod:
    image: mysql:latest
    container_name: mysql-prod
    environment:
      MYSQL_ROOT_PASSWORD: admin
      MYSQL_DATABASE: book
    ports:
      - "3306:3306"
    command: mysqld --character-set-server=utf8mb4 --collation-server=utf8mb4_general_ci
    volumes:
      - ${DOCKER_VOLUME_DIRECTORY:-.}/volumes/mysql-prod:/var/lib/mysql
    restart: always

  nest-app:
    container_name: nest-app
    build:
      context: .
      dockerfile: Dockerfile
    ports:
      - "3000:3000"
    environment:
      NODE_ENV: production
    depends_on:
      - mysql-prod
    restart: always
```

这里 nest-app 的 docker 容器是从 Dockerfile 构建出来的

生成环境连接 mysql 是用容器名，也就是 mysql-prod

所以要改一下：

此外，静态文件默认不会输出到 dist 目录，我们要配置下

改下 nest-cli.json

```
"compilerOptions": {
  "deleteOutDir": true,
  "assets": [
    {
      "include": "../public/**/*",
      "outDir": "dist/public"
```

```
}  
]  
}
```

加一个生产环境用的命令：

```
"docker:prod:up": "docker compose -f docker-compose.prod.yml up -d --build",
```

试一下：

神光的编程秘籍

这样，我们就用 docker compose 实现了生产环境的部署。

代码上传了课程仓库：<https://github.com/QuarkGluonPlasma/ai-agent-course-code>

总结

Agent 开发离不开后端生态，这节我们开始学后端的技术。

首先我们明确了数据库、中间件、业务代码的区分。

数据库是根，存的是原始数据，中间件是特种兵，是完成特定用途的组件，比如缓存、全文检索、消息队列等。

业务代码调度数据库和中间件，实现完整的业务功能，对外提供服务

我们学了 docker 容器怎么跑，volume 数据卷的作用。

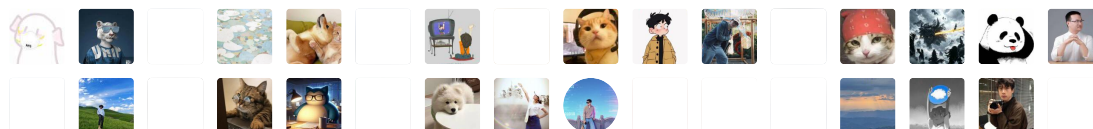
然后写了 Dockerfile，构建出了自己的 docker 镜像，并且基于多阶段构建实现了镜像大小的优化。

学了本地如何用 docker compose 一键启动多个容器

生产环境如何用 docker compose 来部署

后面我们本地开发、生产环境部署，都是基于 Docker Compose 的。

2774 人付费



转型 Agent 全栈工程师：企业级知识库项目 · 目录

上一篇

Agentic RAG：基于 LangGraph 实现大模型
自主决策的 RAG 闭环系统

下一篇

ElasticSearch 全文检索：倒排索引表 + IK
分词器 + BM25 算法

修改于2026年4月25日

